
Probeklausur 6

Grundlagen der Programmierung

Studiengänge: MIB & OMB

Autor: Luis Staudt

Semester: Sommersemester 2026

Bearbeitungszeit: 90 Minuten

Gesamtpunkte: 90

Hinweis: Diese Probeklausur ist bewusst anspruchsvoll gestaltet. Achten Sie besonders auf Schleifengrenzen, Reihenfolge von Anweisungen und die korrekte Behandlung von Randfällen.

Aufgabe	1	2	3	4	Gesamt
Punkte	25	20	20	25	90
Erreichte Punkte					

Aufgabe 1: Bildverarbeitung – drei Schritte kombiniert**(25 Punkte)**

Ein Bitmap werde in einem zweidimensionalen `int`-Array `p` gespeichert. Jede Farbe wird durch eine Zahl vom Typ `int` dargestellt. Die Variable `p[x][y]` steht für die Farbe des Bildes an der Koordinate (x, y) . Der Ursprung des Koordinatensystems liegt links oben, somit enthält `p[0][0]` die Farbe des Pixels in der linken oberen Ecke.

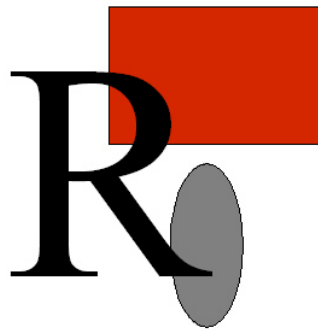
<code>p[0][0]</code>	<code>p[1][0]</code>	<code>p[2][0]</code>	<code>p[3][0]</code>	<code>p[4][0]</code>	<code>p[5][0]</code>
<code>p[0][1]</code>	<code>p[1][1]</code>	<code>p[2][1]</code>	<code>p[3][1]</code>	<code>p[4][1]</code>	<code>p[5][1]</code>
<code>p[0][2]</code>	<code>p[1][2]</code>	<code>p[2][2]</code>	<code>p[3][2]</code>	<code>p[4][2]</code>	<code>p[5][2]</code>
<code>p[0][3]</code>	<code>p[1][3]</code>	<code>p[2][3]</code>	<code>p[3][3]</code>	<code>p[4][3]</code>	<code>p[5][3]</code>
<code>p[0][4]</code>	<code>p[1][4]</code>	<code>p[2][4]</code>	<code>p[3][4]</code>	<code>p[4][4]</code>	<code>p[5][4]</code>

Die Breite des Bildes ist `p.length`, die Höhe ist `p[0].length`. Damit liegt x im Bereich 0 bis `p.length-1` und y im Bereich 0 bis `p[0].length-1`. Der Wert 0 steht für die Farbe schwarz.

Schreiben Sie **eine** Klassenmethode `f` mit Parameter `p` (zweidimensionaler `int`-Array) und ohne Rückgabewert. Die Methode soll **nacheinander** die folgenden drei Schritte ausführen:

- Kreisförmigen Ausschnitt kopieren.** Der Ausschnitt hat den Mittelpunkt $(90, 230)$ und den Radius 45. Jedes Pixel dieses Ausschnitts soll an eine Position kopiert werden, die 165 Pixel weiter **rechts** liegt (Zielmittelpunkt also $(255, 230)$). Es darf angenommen werden, dass Ausschnitt und Zielbereich vollständig im Bild liegen und sich nicht überlappen.
- Eine Form zeichnen (Kreislinie).** Zeichnen Sie um den Mittelpunkt $(171, 95)$ eine schwarze Kreislinie mit innerem Radius 38 und äußerem Radius 55. Alle Pixel, deren Abstand zum Mittelpunkt **zwischen** 38 **und** 55 liegt (jeweils einschließlich), sollen auf 0 gesetzt werden.
- Rahmen schwarz färben.** Färben Sie zum Schluss alle Pixel schwarz, deren Abstand zum Bildrand **kleiner als** 18 Pixel ist (Rahmen der Dicke 18).

Die Reihenfolge ist verbindlich: erst a), dann b), dann c).

*Originalbild**Zielbild nach Aufruf von $f(p)$*

Hinweis. Verwenden Sie das Abstandsquadrat $((x - m_x)^2 + (y - m_y)^2)$ statt einer Wurzel. Für den Ring brauchen Sie *zwei* Bedingungen, für den Rahmen die Oder-Verknüpfung der vier Randbereiche.

Aufgabe 2: Syntaxfehler – Klassen Geometrie, Rechteck und Programm (20 Punkte)

Der Programmcode der drei Klassen `Geometrie`, `Rechteck` und `Programm` enthält mehrere **verschiedene** syntaktische Fehler. Geben Sie zu jedem Fehler die Zeilennummer der Zeile an, in der sich der Fehler befindet, und beschreiben Sie jeweils den Fehler.

```
1 public class Geometrie {
2
3     public double radius;
4
5     public static double flaeche(double r) {
6         return 3.14159 * r * r
7     }
8
9     public static double umfang(double r) {
10        return 2 * 3.14159 * r;
11    }
12
13    public double durchmesser() {
14        return 2 * radius;
15    }
16
17    public static boolean istGross(double r) {
18        if (r > 100)
19            return true;
20    }
21 }
22
23 public class Rechteck {
24
25     public int breite;
26     public int hoehe;
27
28     public static int flaeche(int b, int h) {
29         return b * h;
30     }
31
32     public static int skaliere(int wert, int faktor) {
33         int ergebnis = wert * faktor;
34     }
35
36     public boolean istQuadrat(int b, int h) {
37         return b - h;
38     }
39 }
```

```
40
41 public class Programm {
42
43     public static void zeige(String text) {
44         System.out.println(text);
45     }
46
47     public static void main(String[] args) {
48         double a = Geometrie.flaeche(5.0);
49         zeige(a);
50         double u = Geometrie.umfang();
51         int f = Rechteck.flaeche(4, 6);
52         boolean q = istQuadrat(4, 4);
53         System.out.println(Geometrie.radius);
54         int s = Rechteck.skaliere(3, 2, 1);
55     }
56 }
```

Zeile	Fehler

Aufgabe 3: Programmausgabe – Rekursion und Schleifen**(20 Punkte)**

Zu welcher Ausgabe kommt es, wenn man die Methode `main` der Klasse `X` ausführt?

```
1 public class X {
2
3     public static void zeichne(int n) {
4         if (n <= 0) {
5             System.out.print(".");
6             return;
7         }
8         System.out.print(n);
9         zeichne(n - 1);
10        System.out.print(n);
11    }
12
13    public static void tabelle(int zeilen) {
14        int i = 1;
15        while (i <= zeilen) {
16            int summe = 0;
17            for (int j = 1; j <= i; j++) {
18                summe = summe + j;
19            }
20            System.out.println(i + ":" + summe);
21            i = i * 2;
22        }
23    }
24
25    public static void main(String[] args) {
26        for (int k = 2; k >= 0; k--) {
27            zeichne(k);
28            System.out.println();
29        }
30        tabelle(5);
31        int n = 16;
32        while (n > 1) {
33            System.out.print(n + " ");
34            n = n / 2;
35        }
36        System.out.println();
37    }
38 }
```

Hinweis. `System.out.print` erzeugt keinen, `System.out.println` erzeugt einen Zeilenumbruch.

Empfehlung. Die Methode `zeichne` gibt *vor* und *nach* dem rekursiven Aufruf jeweils eine Zahl aus. In `tabelle` wird `i` nach jeder Zeile *verdoppelt*, in der `while`-Schleife von `main` wird `n` jeweils *halbiert*.

Aufgabe 4: Methode implementieren – zwei sortierte Arrays mischen (25 Punkte)

Schreiben Sie eine statische Methode `mische` mit zwei Parametern `a` und `b` vom Typ `int[]` und einem Rückgabewert vom Typ `int[]`.

Beide Eingabe-Arrays sind bereits **aufsteigend sortiert**. Die Methode soll ein **neues** Array zurückgeben, das alle Elemente aus `a` und `b` enthält und ebenfalls **aufsteigend sortiert** ist (der Mischschritt von *Merge-Sort*). Gleiche Werte dürfen mehrfach vorkommen und bleiben erhalten.

Die Methode soll effizient in **einem Durchlauf** mit zwei Laufindizes arbeiten – also *nicht* erst beide Arrays zusammenkopieren und dann sortieren.

Hinweise. `a.length` liefert die Länge des Arrays `a`; mit `new int[n]` legt man ein neues `int`-Array der Länge `n` an.

Beispiele:

a	b	mische(a, b)
{1,4,7}	{2,3,9}	{1,2,3,4,7,9}
{}	{5}	{5}
{2,2}	{1,2}	{1,2,2,2}
{1,2,3}	{}	{1,2,3}